


Universidad Nacional Autónoma de México

Facultad de Ciencias

Cómputo Concurrente


2027-1

# Tarea 3. Objetos Concurrentes y Volatile


ALUMNOs:

**Martínez Hidalgo Paola Mildred - 319300217**
**Dominguez Sanchez Gustavo Alonso - 418047121**


Fecha de entrega:
**01 de octubre de 2026**


Profesor:
**Gilde Valeria Rodríguez**




# Ejercicios

1. **[15 min/10 puntos]** Contesta de forma breve (máximo 3 líneas por pregunta) lo siguiente:

   *a*) ¿Qué es una especificación secuencial de un objeto?

   Es un conjunto de condiciones que describe cómo es el estado de un objeto cuando ejecuta uno o varios métodos de menra secuencial. En particular, se describe mediante precondiciones (antes de la invocación) y post-condiciones (al terminar). Es el comportamiento secuencial esperado que un algoritmo concurrente debe poder lograr para considerarse correcto.

   *a*) Describe con tus propias palabras, ¿en qué consiste la linealizabilidad?

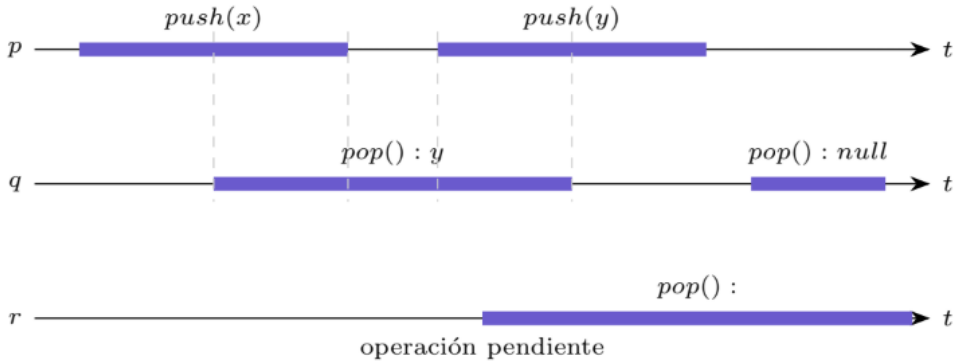
   La linealizabilidad es una fomra de verificar si un algoritmo concurrente cumple las especificaciones secuenciales de un objeto, mediante los puntos de linealización y tomando en cuenta estrictamente el tiempo real o global.

   *a*) Describe con tus propias palabras, ¿cuál es la diferencia entre la consistencia secuencial y la linealizabilidad?

   Aunque ambos evalúan si una implementación o ejecución concurrente es correcta, la diferencia de paradigma radica en que la linealización es estricta con orden global de las operaciones, mientras que la consistencia secuencial basta con mantener el orden local de cada hilo.

2. **[20 min/15 puntos]** ¿La siguiente propiedad es equivalente a decir que un objeto $x$ es *wait-free*? Argumenta por qué.
   **Propiedad.** *Para toda historia infinita $H$ de $x$, cada hilo que toma un número infinito de pasos en $H$, completa un número infinito de llamadas a métodos.* Recordemos que la definición normal de *Wait-Free* nos garantiza que cada llamada a un método termina en un número finito de pasos en ese hilo, independientemente de lo que pase con los demás. Notemos que si un objeto es *Wait-free*, al tomar un hilo infinitos pasos, estará completando un método (en una cantidad finita de pasos), luego otro método y luego otro, así sucesivamente. Como cada uno toma un tiempo finito, un número infinito de pasos obligatoriamente resulta en un número infinito de llamadas a métodos completadas. Sin embargo, si el objeto no fuera *Wait-free* (supongamos solo `Lock-free`), un hilo podría dar infinitos pasos ejecutando eternamente un ciclo. En ese caso, daría un número infinito de pasos, pero completaría cero llamadas a métodos, y no cumpliría la propiedad del enunciado.
   Por lo tanto, la propiedad describe la misma condición de *Wait-free* qué es que se garantiza que todos los hilos terminan incluso tratandose de un tiempo de ejecución infinita.

3. **[30 min/20 puntos]** Considera la siguiente ejecución de una implementación de una pila para tres hilos $p, q, r$ y sobre un solo objeto (de tipo pila).
   La especificación secuencial de una pila es que el método $push(x)$ añade a $x$ al inicio de la pila y el método $pop() : x$ elimina al primer elemento $x$ al inicio de la pila (mantiene LIFO).




*a*) ¿Es linealizable con respecto a una pila? De ser así, incluye una linearización y especifica la *extension* de $H$ y $complete(H')$.

Sea $H$ la ejecución (historia concurrente) de la imagen, se quiere saber si $H$ es linealizable **con respecto a una pila (estructura LIFO)**. La ejecución $H$ es linealizable si existe alguna historia secuencial legal $S$, tal que $S \equiv complete(H)$. Por lo tanto, basta que exhibamos alguna $S$ que cumpla con esto.

Denotaremos que $X$ es concurrente (se traslapa) a $Y$ como $X||Y$, y denotaremos que $X$ precede a (termina antes que) $Y$ como $X < Y$. Recordemos además que:

- Todo par de operaciones concurrentes entre sí, se ordenan entre ellas de la forma que más convenga, es decir, $X||Y$ es equivalente a $Y||X$.
- Las operaciones precedentes entre sí mantienen su ordenen la transformación, es decir que si $X <_H Y$, entonces $X <_J Y$ (con $J$ transformación de $H$).

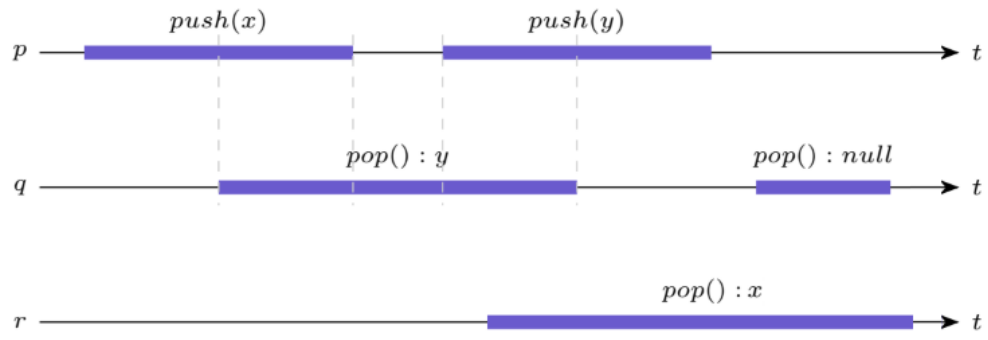
Primero, descubramos que operaciones o tareas son concurrentes y precedentes entre sí. Las reglas que se cumplen en $H$ son:

1. $push(x)$ || $pop() : y$
2. $pop() : y$ || $push(y)$ || $pop() :$
3. $pop() :$ || $pop() : null$
4. $push(x) < push(y)$
5. $push(x) < pop() :$
6. $push(x) < pop() : null$
7. $pop() : y < pop() : null$
8. $push(y) < pop() : null$

Notemos que la historia $H$ tiene una invocación pendiente: la única operación del hilo $r$ ($pop() :$) no tiene respuesta. Para transformar $H$ en una historia secuencial $S$, necesitamos encontrar alguna extensión $H'$ tal que $H' = complete(H)$.

Considerando todas las invocaciones completas de $H$, veamos que en algún punto $x$ y $y$ entran en la pila, y en otro punto $y$ salió de la pila, por la regla 6, 7 y 8, para que $pop() : null$ solo es posible si $x$ salió de la pila. Afortunadamente, la linealizabilidad nos deja decidir si completamos o borramos las invocaciones pendientes, por lo que elegimos

completar la invocación $pop() :$ con una respuesta: $pop() : x$. Por lo tanto, la extensión $H' = complete(H)$ queda como:

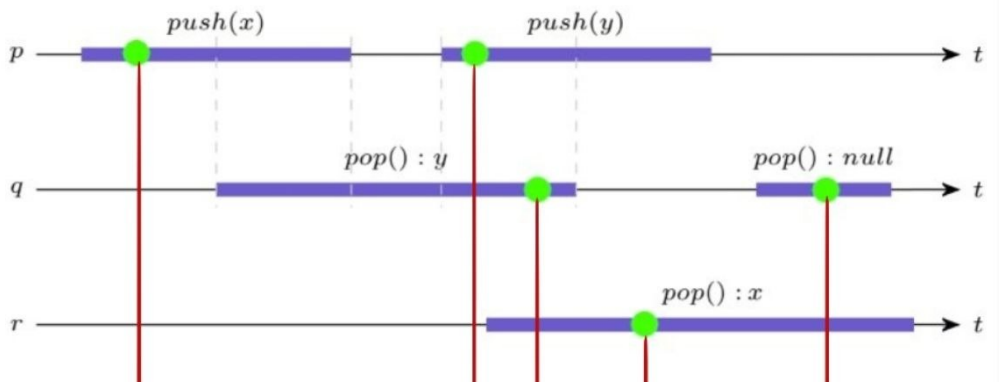



Completando y reordenando (visualmente) nuestras reglas, tenemos:

1. $push(x) \;||\; pop() : y$
2. $push(y) \;||\; pop() : y \;||\; pop() : x$
3. $pop() : x \;||\; pop() : null$
4. $push(x) < push(y)$
5. $push(x) < pop() :$
6. $push(x) < pop() : null$
7. $pop() : y < pop() : null$
8. $push(y) < pop() : null$

Podemos exhibir la siguiente historia secuencial $S$:

$$S = push(x) < push(y) < pop() : y < pop(x) < pop() : null$$




Como $S$ cumple con todas las 8 reglas de nuestras operaciones en $H'$, entonces existe $S$ una historia secuencial legal tal que $S \equiv complete(H)$.

$\therefore$ $H$ es **linealizable** con respecto a una pila.

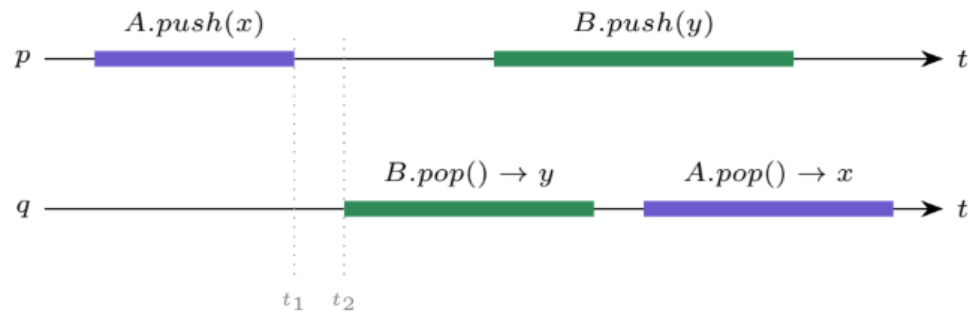
4. **[20 min/20 puntos]** Una propiedad de corrección se define como **componible** (o modular) si la validez del sistema global está garantizada siempre que cada uno de sus objetos componentes sea correcto de manera independiente. En computación concurrente, esta característica es vital para el diseño de sistemas modulares: nos permite diseñar y verificar objetos (como una Pila $A$ y una Pila $B$) por separado, con la certeza de que, al integrarlos en un mismo programa, el sistema resultante preservará dicha propiedad.

   A diferencia de otras condiciones de corrección, la *linealizabilidad* es una propiedad componible. Considera, por ejemplo una ejecución $H$ cuyas proyecciones individuales son linealizables:

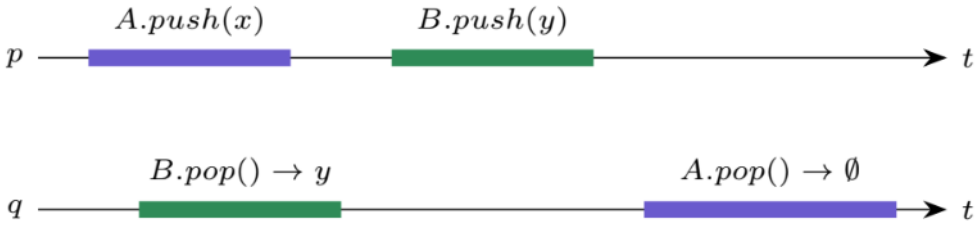
   - $H|A = push(x) \rightarrow pop() : x$
   - $H|B = push(y) \rightarrow pop() : y$

   Debido a que la linealizabilidad es modular, la historia global $H$ es linealizable si existe una historia secuencial legal S que respete el orden de tiempo real de todas las operaciones. En este caso, la historia $S$ resultante es:

   $$S = A.push(x) < B.push(y) < B.pop() : y < A.pop() : x$$




   Considera dos pilas $A$ y $B$. Analiza la siguiente historia $H$:




   *a)* ¿Es $H|A$ secuencialmente consistente? (Pista: Busca un orden global).

Para la subhistoria $H|A$, solo consideramos la proyección sobre la pila $A$, es decir, la operación $A.push(x)$ del hilo $p$, y la operación $A.pop() \rightarrow \varnothing$ del hilo $q$. Dado que la consistencia secuencial permite reordenar operaciones de distintos hilos siempre y cuando se respete el orden local, al ser únicas las operaciones en cada hilo (para $H|A$), podemos construir la historia secuencial $S_A = A.pop() \rightarrow \varnothing < A.push(x)$. Claramente esta secuencia es legal porque si la extracción ocurre antes de la inserción de $x$ (asumiendo una pila vacía al inicio), es correcto devolver $\varnothing$.
Por lo tanto, $H|A$ si es secuencialmente consistente.

*a*) ¿Es $H|B$ secuencialmente consistente?

Nuevamente solo consideramos las operaciones sobre la pila $B$: la operación $B.push(y)$ del hilo $p$, y la operación $B.pop() \to y$ del hilo $q$. Al tratarse de operaciones únicas en su respectivo hilo (se mantiene el orden local), se construye la siguiente historia secuencial (por consistencia secuencial): $S_B = B.push(y) < B.pop() \to y$.
Es claro que es una secuencia legal respecto a una pila, por lo tanto $H|B$ si es secuencialmente consistente.

*a*) ¿Es la historia global $H$ secuencialmente consistente?

Para que $H$ sea secuencialmente consistente únicamente debe respetar el orden local de cada hilo:

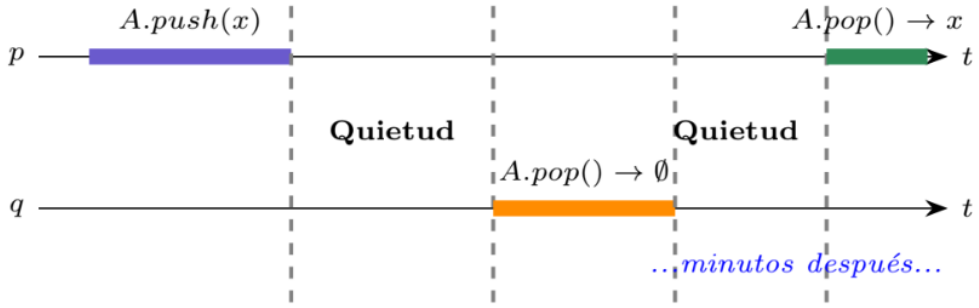
1. $A.push(x) < B.push(y)$ (orden estricto por el hilo $p$)
2. $B.pop() \to y < A.pop() \to \varnothing$ (orden estricto por el hilo $q$)

Tratemos de construir paso a paso una historia legal:

- Tomamos a la regla 2 del hilo $q$ como base:
  $S = B.pop() \to y < A.pop() \to \varnothing$
- Para que la pila $B$ haya extraído al elemento $y$, debió meterlo antes. Agregamos entonces la operación $B.push(y)$ del hilo $p$ al inicio. Al tratarse de hilos separados, por consistencia secuencial podemos insertar esta nueva operación donde convenga:
  $S = B.push(y) < B.pop() \to y < A.pop() \to \varnothing$
- Por la misma consistencia secuencial, al realizar el paso anterior, esto nos obliga a añadir la operación $A.push(x)$ antes de $B.push(y)$ para poder cumplir con el orden obligatorio legal del hilo $p$:
  $S = A.push(x) < B.push(y) < B.pop() \to y < A.pop() \to \varnothing$
- Sin embargo a pesar que esta es una secuencia legal de $S$ (cumple las reglas de orden local), no es una ejecución correcta respecto a pilas: La pila $B$ se mantiene correcta porque primero mete a $y$ y luego lo saca; pero en la pila $A$ se nos dice que mete a $x$ pero cuando se intenta sacar el elemento, $A.pop$ devuelve vacío. Esto es incorrecto.

La situación no cambia si tomamos como base la regla 1 para intentar construir una secuencia legal y correcta $S$. Por lo tanto, $H$ no es secuencialmente consistente.
Más aún, encontramos un ejemplo donde a pesar de que las proyecciones individuales son secuencialmente consistentes, la integración final (historia global) no necesariamente es secuencialmente consistente. Por lo tanto, la consistencia secuencial no es componible.

5. **[15 min/10 puntos]** En una píla asíncrona, el hilo $p$ termina su operación y el sistema entra en un periodo de "inactividad".




*a*) ¿Por qué esta ejecución cumple con Consistencia Eventual pero rompe la Consistencia en la Inactividad?

Esta ejecución cumple con **Consistencia Eventual** porque esta propiedad permite que se observen resultados atrasados, siempre y cuando eventualmente el sistema converja al estado correcto. Esto se cumple cuando $p$ realiza el $pop()$ y recibe $x$, ya que finalmente el sistema termina reflejando el estado final esperado.

Por otro lado, no se cumple la **Consistencia en la Inactividad** porque entre el $push(x)$ y el $pop()$ de $q$ hubo un periodo de quietud. Por lo tanto, el $push(x)$ tuvo que haber quedado completamente antes del $pop()$ de $q$. En consecuencia, cuando $q$ realizó el $pop()$, la pila ya debía contener a $x$ y el resultado correcto era recibir $x$, pero recibió *null*. Como el periodo de quietud impide reordenar el $push(x)$ para colocarlo después del $pop()$, la ejecución no puede corresponder a un orden secuencial válido y se rompe la Consistencia en la Inactividad.

**6. [20 min/25 puntos]** Considera la siguiente clase *Visibility*:

Listing 1: Clase "VisibilityField"

```
class Visibility {
    Shared Variables:
```

```
1     int x=0;
2     volatile Boolean flag=false;
3     void writerThread() {
4         try {
5             Thread.sleep(1000);
6             x = 1;
7             flag = true;
8         } catch(InterruptedException e) {}
9     }
10    void readerThread() {
11        while(!flag){}
12        int y = 100/x;
13    }
```

*a*) ¿Es posible que el método `readerThread()` divida entre cero? Argumenta porqué.

No, en este caso no es posible dividir entre 0. Esto se debe a que `flag` es `volatile`. El hilo que escribe primero cambia `x` a 1 y después cambia `flag` a `true`. Cuando el

sistema observa que la bandera ya está en `true`, también puede observar el cambio anterior de `x`, por lo que `x` ya vale 1 y no puede realizarse una división entre cero.

*b*) Si ambas variables son `volatile`, ¿es posible que el método `readerThread()` divida entre cero? Argumenta porqué.

No, en este caso tampoco es posible, aunque ambas sean volatile al igual que en la respuesta anterior primero se cambiaría el valor de x a 1 y ya después flag a true, por lo que de igual forma no se puede dividir entre 0.

*c*) Si ninguna de las dos variables es `volatile`, ¿es posible que el método `readerThread()` divida entre cero? Argumenta porqué.

Para este caso si es posible que se pueda dividir entre 0, como ya ninguna variable es volatile el sistema puede pasar la bandera a true antes de poner x=1 porque ya se trata de dos instrucciones independientes, es decir, el sistema ve la bandera levantada o en true y sale del ciclo, al salir puede encontrar x=0, así que en este caso si se puede dividir entre 0. Igual otro caso que podemos encontrar es que los valores estén desactualizados, por lo que al llegar a la división puede que x se encuentre con el valor de 0.